INDUSTRIAL ORIENTED MINI PROJECT REPORT ON

ROAD-The Road Event Awareness Dataset for Autonomous Driving

in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering (DATA SCIENCE)

Submitted by

D SRISHANTH 23B81A6752

A PRANEETH 23B81A6728

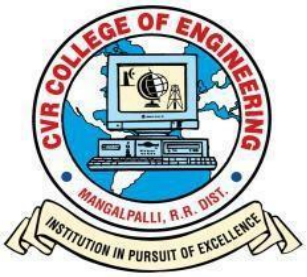
M SAI VIGNESH 23B81A6741

Under the guidance of

Dr. Yasmeen

Sr. Assistant professor

CSE(Data Science)



DEPARTMENT OF CSE (Data Science)

CVR COLLEGE OF ENGINEERING

(***An Autonomous institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad***)

Vastunagar, Mangalpalli (V), Ibrahimpatnam (M), Ranga Reddy (D), Telangana- 501 510

APRIL 2026

CVR COLLEGE OF ENGINEERING

(*An Autonomous institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad*)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Rangareddy (D), Telangana- 501 510

COMPUTER SCIENCE AND ENGINEERING(Data Science)

CERTIFICATE

This is to certify that the Industry Oriented Mini Project report entitled **"ROAD –The Road Event Awareness Dataset for Autonomous Driving"** bonafied record of work carried out by **D. Srishanth (23B81A6752), A. Praneeth (23B81A6728)** and **M. Sai Vignesh (23B81A6741)** submitted to **Dr. Yasmeen, Associate Professor** for the requirement of the award of **Bachelor of Technology** in **CSE (Data Science)** to the CVR College of Engineering, affiliated to Jawaharlal Nehru Technological University, Hyderabad during the year 2025-2026.

Project Guide
Dr. Yasmeen
Sr. Assistant Professor
Department of CSE(DS)

Project Coordinator
Dr. Shaik Janbhasha
Associate Professor
Department of CSE(CS)

Head of the Department
Dr. S. V. Suryanarayana
Professor & Head
Department of CSE(DS)

External Examiner

CVR COLLEGE OF ENGINEERING

(*An Autonomous institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad*)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M) Rangareddy (D),
Telangana- 501 510

DECLARATION

We hereby declare that the Industry Oriented Mini Project report entitled **"ROAD – The Road Event Awareness Dataset for Autonomous Driving"** is an original work done and submitted to Computer Science and Engineering (Data Science) Department, CVR College of Engineering, affiliated to Jawaharlal Nehru Technological University Hyderabad in partial fulfilment for the requirement of the award of Bachelor of Technology in Computer Science and Engineering (Data Science) and it is a record of bonafide project work carried out by us under the guidance of **Dr Yasmeen** Sr. Assistant Professor, Department of CSE (Data Science).

We further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree in this Institute or any other Institute or University.

Signature of the Student
D Srishanth
23B81A6752

Signature of the Student
A Praneeth Reddy
23B81A6728

Signature of the Student
M Sai Vignesh
23B81A6741

Date:

Place:

ACKNOWLEDGEMENT

We are thankful for and fortunate enough to get constant encouragement, support, and guidance from all **Teaching staff of CSE (Data Science) Department** which helped us in successfully completing this Industry Oriented Mini Project.

We are extremely thankful to our internal guide **Dr Yasmeen,** Sr.Assistant Professor, Department of CSE (Data Science) for providing support and guidance, which made us to complete the Industry Oriented Mini Project.

We thank **Dr. Shaik Janbhasha**, Project Review Committee members for their valuable guidance and support which helped us to complete the Industry Oriented Mini Project Work successfully.

We thank Professor, Head of the Department CSE (Data Science), **Dr**. **S. V. Suryanarayana,** for giving us all the support and guidance, which made us to complete the Industry Oriented Mini Project duly.

Our gratitude also express heartful thanks to **Dr. Lakshmi H. N**, Associate Dean, ET for providing us an opportunity and extending support and guidance.

We thank our Vice-Principal **Prof. L. C. Siva Reddy** for providing excellent computing facilities and a disciplined atmosphere for doing our work.

We wish a deep sense of gratitude and heartfelt thanks to our Principal **Dr. Rama Mohan Reddy** and the **Management** for providing excellent lab facilities and tools.

ABSTRACT

Road accidents remain one of the leading causes of fatalities worldwide, with human error accounting for over 90% of incidents. Factors such as distracted driving, delayed reaction times, poor visibility, and lack of situational awareness significantly contribute to these accidents. With the rapid advancement of artificial intelligence and computer vision, there is a growing need for intelligent systems that can assist drivers in real time and reduce the risk of collisions. This project presents ROAD (Real-time Object Analysis for Driving), an intelligent video-based road safety detection system that leverages the YOLOv8 deep learning model for high-speed and accurate object detection, along with centroid-based multi-object tracking, lane position classification, and a weighted risk assessment algorithm. The system is designed to function efficiently in dynamic traffic environments, making it suitable for both research and real-world applications.

The system processes dashcam video footage frame by frame to detect various road objects such as cars, pedestrians, trucks, motorcycles, and bicycles with high precision. It further analyzes object motion by comparing bounding box size variations across consecutive frames to determine whether objects are approaching or moving away from the vehicle. Additionally, it classifies each detected object into lane regions (left, centre, right) using spatial positioning techniques. A composite risk score is then computed based on multiple parameters including object proximity, motion direction, type of object, and lane position. Based on this score, risk levels are categorized as LOW, MEDIUM, HIGH, or CRITICAL, enabling proactive hazard warnings and timely decision-making. The system outputs an annotated video with colour-coded bounding boxes, motion indicators, lane boundaries, and a real-time summary dashboard displaying detected objects and their respective risk levels.

TABLES OF CONTENTS

Chapter No.		Contents	Page No.
		Acknowledgement	iii
		Abstract	iv
		List of Tables	vi
		List of Symbols	vii
		List of Figures	viii
		List of Abbreviations	ix
1		**Introduction**	**1-6**
	1.1	Motivation	2
	1.2	Project Statement	3
	1.3	Project Objectives	3-4
	1.4	Project Report Organization	5-6
2		**Literature Survey**	**7-8**
	2.1	Existing Work	7
	2.2	Limitations of Existing Work	8
3		**Software & Hardware specifications**	**9-13**
	3.1	Functional Requirements	9-10
	3.2	Non-Functional Requirements	10-11
	3.3	Software requirements	12
	3.4	Hardware requirements	13
4		**Proposed System Design**	**14-23**
	4.1	Proposed methods	14-15
	4.2	Class Diagram	16-18
	4.3	Use Case Diagram	19-21
	4.4	Activity Diagram	22-24
	4.5	Sequence Diagram	25-26
	4.6	System Architecture	27-29
	4.7	Technology Description	30-31
5		**Implementation & Testing**	**32-40**
	5.1	Implementation	32-34
	5.2	Frontend Implementation	35-36
	5.3	Testing and Results	37-38
	5.4	Summary Table of Test Execution	39-40
6		**Conclusion & Future Scope**	**41-42**
		References	**43**
		Appendix: (Sample Source Code)	**44-48**

LIST OF TABLES

Table No.	Title	Page No.
3.3	Software Requirements	12
3.4	Hardware Requirements	13
5.4	Summary Table of Test Execution	39

LIST OF SYMBOLS

Symbol	Description
YOLOv8	State-of-the-art CNN model (Ultralytics) used for real-time object detection.
.pt	PyTorch model file extension used for the YOLOv8 weights (e.g., yolov8n.pt).
bbox	Bounding box coordinates [x1, y1, x2, y2] representing an object's location.
centroid	The geometric centre (cx, cy) of a bounding box, used for object tracking.
EMA	Exponential Moving Average is used to smooth risk scores and prevent flickering.
TTC	Time-to-Collision: a calculation determining the seconds until a potential impact.
H.264	Video compression standard used to re-encode analysed footage for browser playback.
cv2	The OpenCV library namespace is used for image processing and visualisation.
LANE_CENTER_MAX	Configuration constant defining the percentage boundary for the centre lane.
RISK_WEIGHTS	A dictionary of coefficients (distance, motion, lane) that sum to 1.0.
APPROACHING	A motion state is triggered when an object's bounding box area increases over time.
DEPARTING	A motion state is triggered when an object's bounding box area decreases over time.
multi_object_boost	A logic flag that increases risk when multiple hazards are in the centre lane.
argparse	Python module used to handle command-line inputs like --video or --no- save.
.css	Cascading Style Sheets are used to define the dark-themed UI of the dashboard.
.js	JavaScript file handling the interactive pipeline animations on the frontend.
subprocess	Python module used to link the analysis script with the website automation script.

LIST OF FIGURES

Figure No.	Title	Page No.
4.2	Class Diagram	18
4.3	Use Case Diagram	21
4.4	Activity Diagram	24
4.5	Sequence Diagram	26
4.6	System Architecture	27
5.2.1	Home Page	35
5.2.2	System Module	35
5.2.3	Pipe Line	36
5.2.5	Teck Stack	36
5.4.1	Output	40
5.4.2	Output	40

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ROAD	Real-time Object Analysis for Driving Safety
ADAS	Advanced Driver-Assistance Systems
ITS	Intelligent Transportation Systems
YOLO	You Only Look Once (Object Detection Algorithm)
CNN	Convolutional Neural Network
COCO	Common Objects in Context (Dataset used for YOLO training)
FPS	Frames Per Second
BBox	Bounding Box
EMA	Exponential Moving Average
TTC	Time-to-Collision
BGR	Blue-Green-Red (Color space used by OpenCV)
CUDA	Compute Unified Device Architecture (Parallel computing by NVIDIA)
CPU	Central Processing Unit
GPU	Graphics Processing Unit
OpenCV	Open Source Computer Vision Library
DOM	Document Object Model (Used in script.js for UI updates)
IOMP	Integrated Overall Management Plan (or Project-specific internal naming)
H.264	Advanced Video Coding (Video compression standard)

CHAPTER 1

INTRODUCTION

The rapid increase in vehicular traffic has resulted in a substantial rise in road accidents worldwide. According to the World Health Organization, road traffic injuries are among the leading causes of death globally, highlighting the urgent need for effective safety measures and intelligent monitoring systems to reduce such incidents.

In recent years, computer vision and deep learning have emerged as powerful tools for enhancing road safety. Object detection models like YOLO (You Only Look Once) enable fast and accurate identification of various road users, while tracking algorithms help analyze their movement patterns over time. By integrating these capabilities with lane detection and risk assessment techniques, it becomes possible to develop a comprehensive and intelligent road safety analysis framework.

This project develops a real-time road safety detection system that: Detects road objects (vehicles, pedestrians, cyclists) using YOLOv8 Tracks objects across video frames to determine motion direction Classifies objects by lane position for spatial awareness Computes risk scores based on proximity, motion, lane, and object type Generates annotated video output with visual risk indicators The system is designed to work with standard dashcam footage and can serve as a foundation for ADAS development, traffic analysis, and autonomous driving research.

In recent years, advancements in intelligent transportation systems (ITS) and advanced driver-assistance systems (ADAS) have opened new possibilities for reducing road accidents. These systems aim to improve driving safety by providing real-time alerts, monitoring road conditions, and assisting in decision- making. Among the various technologies used, computer vision and deep learning have emerged as powerful tools for analyzing visual data from road environment.

1.1 Motivation

Road safety has become a critical global issue due to the continuous rise in vehicular traffic and the increasing number of road accidents. Despite advancements in transportation infrastructure, a large percentage of accidents still occur due to human error, such as lack of attention, delayed reaction, or poor judgment.

With the rapid development of computer vision and deep learning technologies, there is a significant opportunity to enhance road safety through automated analysis of traffic environments. Modern object detection models, such as YOLOv8, provide the capability to identify multiple road users accurately and in real time. However, simple detection alone is not sufficient to prevent accidents.

Existing driver-assistance systems primarily focus on isolated functionalities such as lane detection or collision warnings. These systems often lack a unified approach that integrates detection, tracking, lane awareness, and risk assessment into a single framework.

The motivation behind this project is to design a real-time road safety detection system that goes beyond traditional methods by combining multiple analytical components. By integrating object detection, motion tracking, lane classification, and a weighted risk scoring mechanism, the system aims to evaluate the driving environment more intelligently and provide proactive warnings. Such a system can assist drivers in identifying potential hazards early, thereby reducing the likelihood of accidents.

1.2 Problem Statement

Current road safety systems face several challenges:Lack of Real-time Analysis: Many existing systems perform offline analysis and cannot provide instantaneous hazard warnings during driving.

Limited Motion Understanding: Standard object detection identifies objects in individual frames but does not determine whether objects are approaching or moving away.

No Integrated Risk Assessment: Detection systems typically report raw detections without contextual risk evaluation combining proximity, motion trajectory, lane position, and object vulnerability.Complexity of Deployment: Many research systems require expensive sensor suites (LiDAR, radar) and are impractical for consumer-grade implementation.

There is a need for a lightweight, vision-only system that can process standard dashcam video to detect road objects, understand their motion dynamics, and predict collision risk factors in real time.

1.3 Project Objectives

The primary objective of the ROAD (Real-time Object Analysis for Driving Safety) system is to develop an intelligent vision-based solution that enhances driving safety through real-time analysis of road environments. The project aims to detect various road objects such as vehicles, pedestrians, cyclists, and motorcycles using a deep learning-based object detection model. It also focuses on tracking multiple objects across consecutive frames to understand their movement and determine whether they are approaching, departing, or stationary. Another objective is to classify detected objects

based on their lane position to identify potential hazards directly in the vehicle's path. The system further aims to compute a weighted risk score using factors such as distance, motion, lane position, and object type, and categorize the risk into different levels for effective warning. Additionally, the project intends to provide intuitive visual feedback through color-coded bounding boxes, motion labels, and annotated video output. The modular architecture ensures scalability and supports future enhancements, making the system suitable for advanced driver-assistance systems and autonomous driving research.

The project also aims to create a lightweight and cost-effective safety system that works with standard dashcam footage without requiring expensive sensors such as LiDAR or radar. It focuses on designing a modular pipeline that integrates object detection, tracking, lane classification, and risk assessment into a single framework. Another objective is to generate real-time alerts and annotated visual outputs that help drivers understand surrounding traffic conditions quickly. The system is intended to improve situational awareness by continuously monitoring the driving environment.

1.4 Project Report Organization

This report is systematically structured into six comprehensive chapters, each addressing a key phase in the lifecycle of the ROAD system:

Chapter 1: Introduction Begins by presenting a detailed overview of the global road safety crisis and the potential of Intelligent Transportation Systems (ITS). It emphasizes the importance of real-time hazard detection using artificial intelligence to mitigate accidents. It outlines the project's core objectives—detecting, tracking, and assessing risk—and the relevance of using the YOLOv8 architecture. This chapter establishes the foundation for bridging the gap between raw video data and actionable driver-assist alerts.

Chapter 2: Literature Review Explores existing research in Advanced Driver-Assistance Systems (ADAS). It surveys previous methodologies, ranging from traditional sensor-based detection (Radar/LiDAR) to early deep learning techniques. This chapter critically examines the limitations of prior vision-only methods—such as high latency or lack of spatial awareness—and justifies the need for our improved approach using a weighted multi-factor risk engine and centroid-based motion tracking.

Chapter 3: Software and Hardware *Specifications* Outlines the technical environment required to implement the system. It details functional requirements—such as frame extraction, lane classification, and risk scoring—as well as non-functional aspects like real-time performance (FPS), system reliability, and UI responsiveness. It enumerates specific tools used, including Python, OpenCV, PyTorch, and Ultralytics, alongside hardware requirements such as GPU acceleration for near-real-time inference.

Chapter 4: Proposed System Design Delves into the technical framework of the ROAD project. It provides an in-depth explanation of the modular pipeline, from YOLOv8

inference to the Exponential Moving Average (EMA) smoothing of risk scores. The chapter includes multiple UML diagrams to visually represent the system's static structure and dynamic interactions. It explains the logic behind the Lane Detector and Risk Assessor integration, showing how data flows from a raw video frame to a color-coded safety dashboard.

Chapter 5: Implementation and Testing Focuses on the practical aspects of building the safety system. It describes the use of the COCO dataset for pre-training, data preprocessing techniques, and the development of the custom tracking and risk logic. The chapter elaborates on the development of the web-based dashboard using HTML/CSS/JS, which allows for a visual showcase of analyzed footage. Results and discussions on detection accuracy and risk-scoring precision are presented, alongside specific test cases that validate the system's robustness in complex traffic scenarios.

Chapter 6: Conclusion and Future *Scope* Summarizes the project outcomes and evaluates its success in providing a vision-only safety solution. It reflects on the effectiveness of the Multi-Object Boost logic and the intuitive nature of the visual alerts. The chapter discusses current limitations, such as fixed lane boundaries, and proposes future enhancements like Deep-Learning Lane Detection, Time-to-Collision (TTC) precision improvements, and edge-device deployment for real-vehicle integration.

CHAPTER 2

LITERATURE SURVEY

2.1 Existing Work

Traditional driver assistance systems rely heavily on manual observation by drivers, which increases the chances of human error and delayed reaction to road hazards [7].

Many existing object detection systems focus only on identifying objects in individual frames without analyzing motion or predicting potential risks [8].

Some advanced safety systems depend on expensive sensors such as LiDAR and radar, making them unsuitable for low-cost or consumer-level deployment [9].

Existing solutions often lack integrated risk assessment that combines distance, motion, lane position, and object type into a unified safety metric [5].

Most traditional approaches perform offline processing and are unable to provide real-time hazard detection during active driving [2].

Conventional lane detection methods may fail in complex scenarios such as curved roads, shadows, and poor lighting conditions, reducing system reliability [3].

Current tracking techniques in basic systems struggle with object occlusion and identity switching, affecting motion understanding accuracy [4].

Many available road safety systems do not provide intuitive visual feedback or color-coded warnings for drivers to quickly interpret risk levels [1].

2.2 Limitations of Existing System

Existing road safety systems primarily depend on manual driver awareness, which increases the possibility of delayed reactions and human error in critical situations [7].

Many existing object detection approaches analyze frames independently and fail to capture motion patterns, limiting their ability to predict potential collisions [8].

Several advanced driver assistance solutions rely on costly hardware such as LiDAR and radar sensors, making them less accessible for low-cost implementations [9].

Traditional systems often lack a comprehensive risk assessment mechanism that combines multiple factors such as distance, motion, lane position, and object type [5].

Most existing approaches are designed for offline processing and cannot provide real-time alerts necessary for dynamic driving environments [2].

Conventional lane detection techniques struggle in challenging conditions such as poor lighting, shadows, rain, or curved roads, affecting accuracy [3].

Basic tracking methods used in earlier works face difficulties in handling occlusions and maintaining object identity across frames [4].

Existing solutions frequently do not offer intuitive visual feedback or color-coded warning systems, reducing their effectiveness in assisting drivers [1].

CHAPTER 3

SOFTWARE AND HARDWARE SPECIFICATIONS

The development of the ROAD (Real-time Object Analysis for Driving Safety) system required a robust technical environment capable of handling high-speed video processing and deep learning inference. A primary goal was to ensure the system remains user-friendly by automating complex risk calculations and providing an intuitive visual interface, reducing the cognitive load on the driver.

3.1 Functional Requirements

Functional requirements define the specific behaviours and services the system must provide. For a vision-based ADAS (Advanced Driver-Assistance System) leveraging YOLOv8 and custom tracking logic, these include:

1. **Real-Time Object Detection:**

- The system must identify road-relevant entities (Cars, Pedestrians, Trucks, Buses, Motorcycles, Bicycles) from a live or recorded video stream using the YOLOv8 nano model.

2. **Object Tracking & ID Persistence:**

- The system must assign unique IDs to detected objects and maintain these identities across consecutive frames using a centroid-based tracking algorithm.

3. **Motion State Analysis:**

- By analysing the change in bounding box area over time, the system must classify object motion as APPROACHING, DEPARTING, or STATIONARY.

4. **Spatial Lane Classification:**

- The system must divide the camera's field of view into LEFT, CENTER, and RIGHT zones and dynamically assign every detected object to a lane based on its horizontal

coordinates.

5. **Multi-Factor Risk Scoring:**

- The system must calculate a numerical risk score (0.0–1.0) based on weighted parameters: Proximity (Size), Motion State, Lane Position, and Object Type.

6. **Temporal Smoothing:**

- Implement **Exponential Moving Average (EMA)** logic to smooth risk scores across frames, preventing visual flickering and ensuring stable alert levels.

7. **Dynamic Visual Feedback:**

- The system must render color-coded bounding boxes (Green/Yellow/Orange/Red) and a real-time "Safety Dashboard" overlay that updates based on the current highest hazard.

8. **Automated Web Deployment:**

- Provide a one-click integration script (run_and_update_website.py) that analyzes video, re-encodes it for browser compatibility (H.264), and launches a web-based dashboard for review.

3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints that ensure the system is reliable, efficient, and secure.

1. **Performance & Latency:**

- **Inference Speed:** The system should target near-real-time processing (~20-30 FPS on GPU-enabled systems) to ensure alerts are relevant to current road conditions.
- **Latency:** The delay between frame capture and risk assessment display must be minimized to provide a responsive safety environment.

2. **Usability:**

- **Intuitive UI:** The visual annotations must be "glance-able," using standard traffic light colours to communicate danger levels without requiring the driver to read text.

- **Accessibility:** The web-based dashboard must be compatible with modern browsers (Chrome, Firefox, Safari) using standard HTML5/CSS3.

3. **Reliability:**

- **Robustness:** The system must handle "disappeared" frames (up to 30 frames by default) without losing object identity, ensuring tracking continuity in case of momentary occlusions.
- **Error Handling:** The system should gracefully handle missing video files or incompatible codecs, providing clear terminal feedback to the user.

4. **Scalability:**

- **Model Flexibility:** The architecture should allow for swapping the YOLOv8 nano model for larger versions (Small, Medium, Large) if higher accuracy is required on more powerful hardware.
- **Modular Design:** Components like the Risk Assessor or Lane Detector should be independent modules to allow for future logic updates without refactoring the entire codebase.

5. **Maintainability:**

- **Centralized Configuration:** All tuneable parameters (weights, thresholds, colors) must be stored in a single config.py file to facilitate easy experimentation and tuning.
- **Version Control:** Utilize Git to manage changes in detection logic, model weights, and the frontend codebase.

3.3 Software Requirements

Category	Technology	Version	Description
Backend	Python	3.4.4	Core application logic and pipeline orchestration
	OpenCV	4.8+	Video processing and frame manipulation
Frontend	OpenCV GUI	Built-in	Display annotated video output
	HTML5	Latest	Structure of web interface
	CSS3	Latest	Styling and layout design
	JavaScript (ES6)	ES6	Client-side interactivity and dynamic behavior
Machine Learning	YOLOv8 (Ultralytics)	8.0+	Real-time object detection
	PyTorch	2.0+	Deep learning framework for YOLOv8
Development Tools	VS Code	Latest	Code development and debugging
	pip	21.0+	Python package management

3.4 Hardware Requirements

Component	Minimum	Recommended	Notes
Processor	Intel i5 / AMD Ryzen 5	Intel i7 / AMD Ryzen 7	Multi-core CPU improves ML inference performance
RAM	8GB	16GB	Sufficient memory for smooth performance during emotion analysis
Storage	256GB SSD	512GB+ SSD	SSD improves model loading, database access, and app responsiveness
Operating System	Windows 10/11, macOS, Linux	Linux (Ubuntu 20.04+)	Linux offers better ML and Python environment stability
Display	720p resolution	Full HD (1080p) or higher	Better clarity for debugging UI/camera interaction

CHAPTER 4

PROPOSED SYSTEM DESIGN

4.1 Proposed Method

The proposed system follows a modular, pipeline-based architecture that processes video frames sequentially through multiple stages. The processing pipeline begins with the input video, followed by object detection, motion tracking, lane classification, risk assessment, and finally annotated output generation. Each frame passes through the complete pipeline, and the results are accumulated across frames to enable temporal analysis, particularly for tracking and motion detection.

Input Video → Detection → Tracking → Lane Classification → Risk Assessment → Annotated Output

Key Features

The system performs object detection using the YOLOv8 nano model trained on the COCO dataset, where 80 classes are filtered to six road-relevant categories. Motion tracking is implemented using a centroid-based tracking approach combined with bounding box area change analysis to determine object movement. Lane detection is performed using frame zone division, where the frame is split into left, center, and right regions using a 35%, 30%, and 35% division respectively. Risk assessment is carried out using a weighted multi-factor scoring method that considers distance (35%), motion (30%), lane position (20%), and object type (15%). Visualization is implemented using OpenCV-based annotations with color-coded risk overlays to highlight potential hazards.

The risk score is calculated using a weighted combination of multiple factors, including distance, motion, lane position, and object type. The formula is defined as:

Risk Score = (w_1 × Distance_Factor) + (w_2 × Motion_Factor) + (w_3 × Lane_Factor) + (w_4 × Object_Factor)

The distance factor ($w_1 = 0.35$) is determined based on the vertical position of the object in the frame, where objects appearing lower in the frame are considered closer and assigned higher risk. The motion factor ($w_2 = 0.30$) assigns values based on object movement, with approaching objects given 1.0, stationary objects 0.5, and departing objects 0.1. The lane factor ($w_3 = 0.20$) assigns a value of 1.0 to objects in the center lane and 0.5 to objects in the left or right lanes. The object factor ($w_4 = 0.15$) assigns risk weights based on object type, where pedestrians are given 1.0, bicycles 0.8, motorcycles 0.7, buses and trucks 0.6, and cars 0.5.

The calculated risk score is categorized into four levels for decision-making. A low-risk level corresponds to a score between 0 and 25 percent and is represented using green color, indicating normal driving conditions. A medium-risk level corresponds to a score between 25 and 50 percent and is represented using yellow color, suggesting closer monitoring. A high-risk level corresponds to a score between 50 and 75 percent and is represented using orange color, indicating preparation for braking. A critical-risk level corresponds to a score between 75 and 100 percent and is represented using red color, indicating an immediate hazard warning.

4.2 Class Diagram

To define the static blueprint of the system by outlining the classes, their internal attributes, and the methods that govern object interaction. The diagram illustrates a modular architecture where a central Main class coordinates with specialized utility classes like Detector, Centroid Tracker, Lane Detector, and Risk Assessor. It highlights the use of the YOLOv8 model for inference and OpenCV for the Visualizer output.

By separating concerns into distinct classes, the system achieves high maintainability. For instance, the Risk Assessor class can be updated with new scoring logic independently of the Detector class. This structure ensures that data flows logically from raw frames to analyzed risk metadata.

Classes:

ROADConfig:

Stores configuration parameters such as model path, target classes, and thresholds. Provides centralized control for system tuning and behavior settings.

Detection:

Represents detected objects returned by the YOLO model. Contains bounding box, class label, confidence score, and centroid information.

TrackedObject:

Maintains information about tracked objects across video frames. Stores unique object ID and centroid to analyze motion over time.

LaneDetector:

Divides the frame into left, center, and right zones. Determines the lane position of each detected object.

CentroidTracker:

Tracks objects using centroid-based matching between frames. Handles object registration, updating, and disappearance logic. .

RiskAssessor:

Calculates risk score based on distance, motion, lane, and object type. Categorises risk into LOW, MEDIUM, HIGH, and CRITICAL levels. .

Visualizer:

Draws bounding boxes, labels, and lane boundaries on frames. Displays real-time risk indicators and statistics dashboard. .

ROADSystem (Main Controller): .

Coordinates all modules in the processing pipeline. Manages detection, tracking, lane classification, and risk evaluation flow.

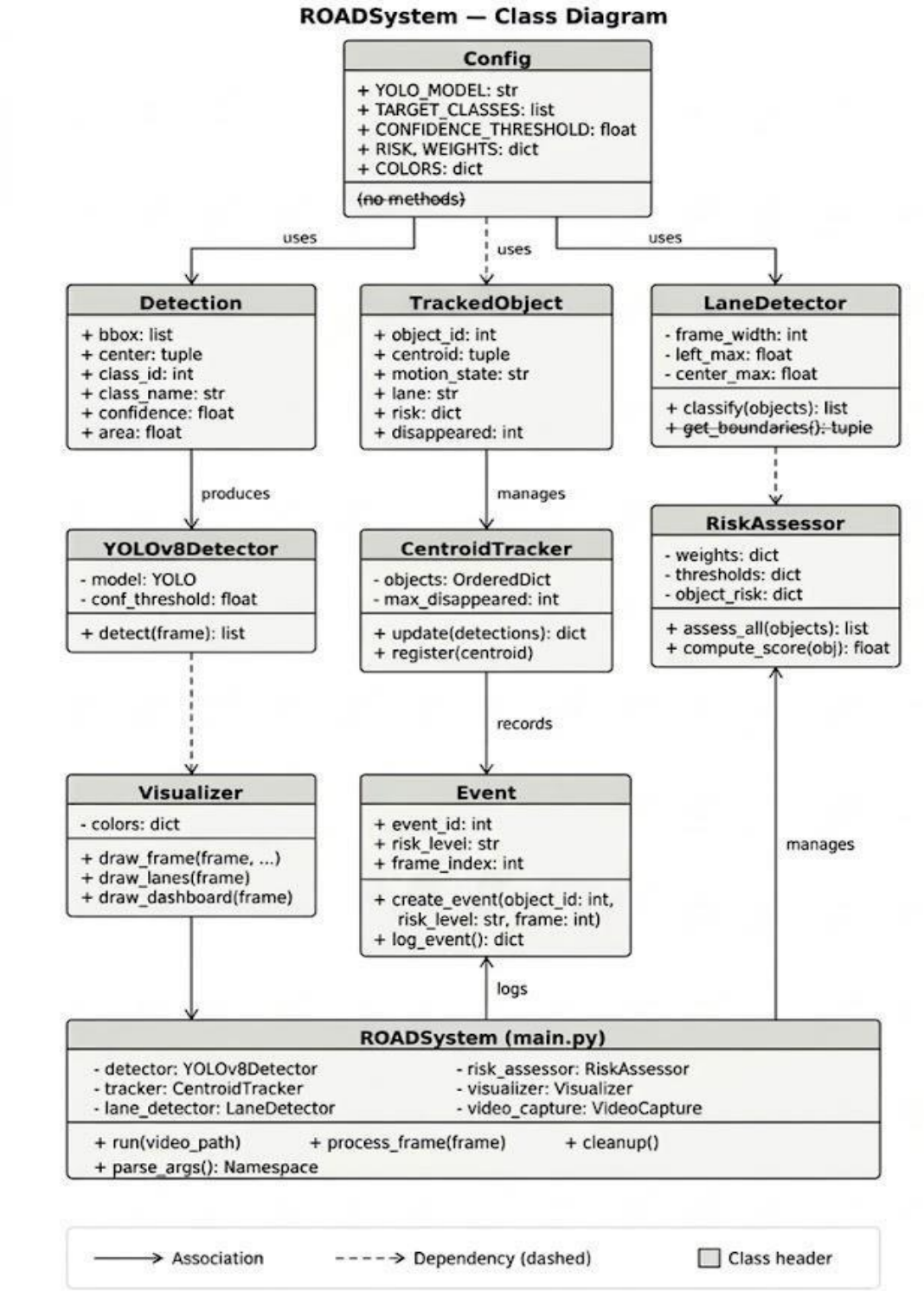


Fig 4.2 Class Diagram

4.3 Use Case Diagram

To define the functional scope of the system and identify how the external actor (User/Driver) interacts with the automated detection pipeline.**:** The diagram identifies the **User** as the primary actor who initiates the Video Input process. The system then executes several core use cases, including Object Detection, Motion Tracking, Lane Classification, and Risk Assessment, which are essential for generating the final safety feedback.

It visually maps the dependency between raw data input and the resulting safety outputs. Specifically, it shows that the system's primary goal is to provide Visual Alerts and an Updated Dashboard, ensuring the user receives real-time information regarding road hazards and collision risks.

Components:

Driver:

Represents the end user who interacts with the system during driving. Receives visual warnings and hazard alerts from the road safety system.

Admin:

Represents the system administrator responsible for monitoring performance. Manages system operation, video input, and overall functionality.

Provide Upload Video:

Allows the user to input dashcam or recorded driving video. Acts as the starting point for processing and analysis.

Detect Road Objects:

Identifies vehicles, pedestrians, and cyclists using object detection. Uses deep learning model to extract bounding boxes and class labels.

TrackObjectMotion:

Tracks detected objects across consecutive frames. Determines whether objects are approaching, departing, or stationary.

ClassifyLanePositions:

Divides the frame into left, center, and right zones. Assigns each detected object to its respective lane.

ComputeRiskScore:

Calculates risk level using distance, motion, lane, and object type. Produces a weighted score indicating collision probability.

DisplayRiskLevels:

Shows LOW, MEDIUM, HIGH, and CRITICAL risk categories. Uses color-coded visual indicators for easy understanding.

SaveAnnotatedVideo:

Stores processed video with detection and risk overlays. Allows future analysis and documentation of results.

ReceiveHazardAlerts:

Provides warnings when high-risk objects are detected. Helps drivers take preventive actions immediately.

MonitorDetection&TrackingPerformance:

Allows admin to observe system accuracy and efficiency. Heliates debugging and performance improvement of the model.

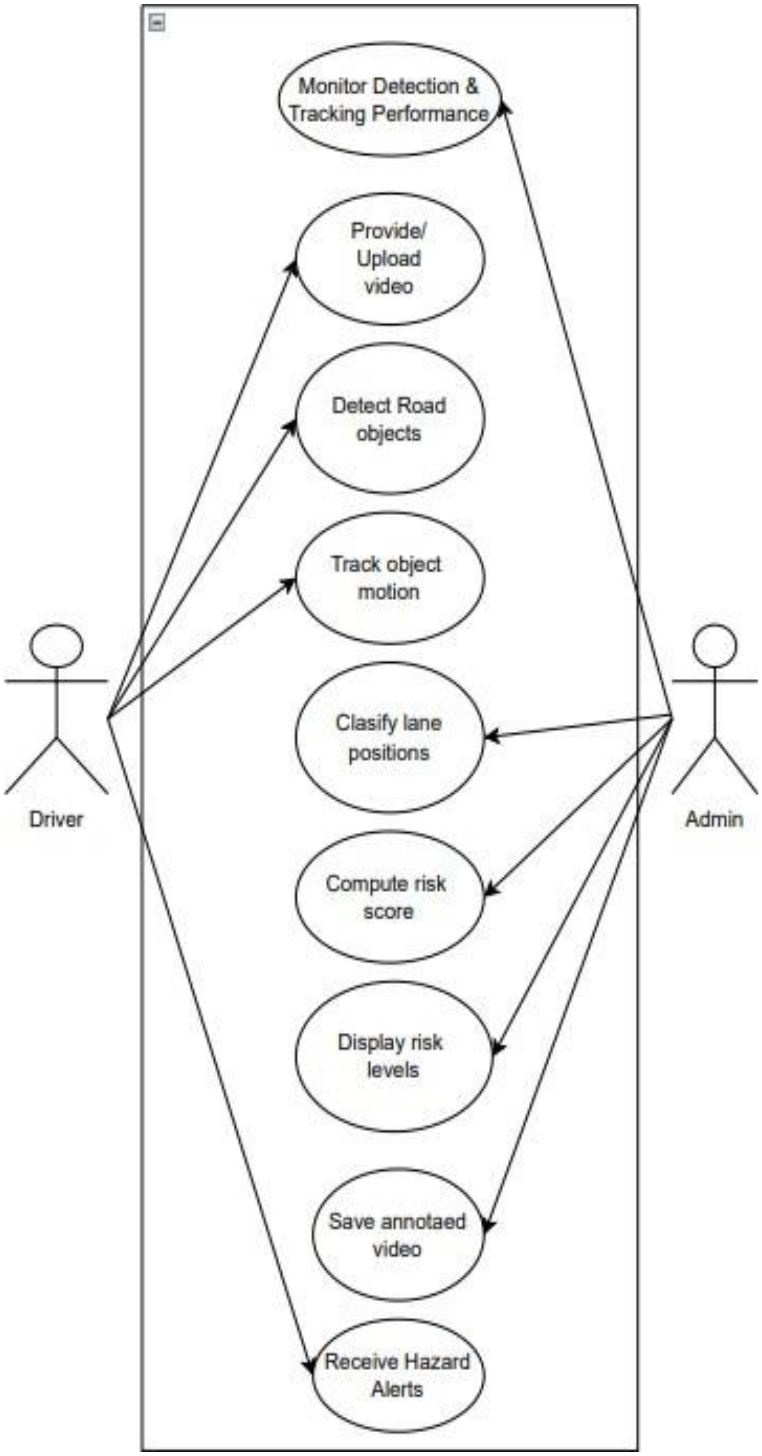


Fig 4.3 Use Case Diagram

4.4 Activity Diagram

To illustrate the static structure and object-oriented design of the system by showing the relationships between code modules. The diagram displays the core classes including Detector, Centroid Tracker, Lane Detector, Risk Assessor, and Visualizer. It shows how the Main controller aggregates these specialized classes to process video data.

It defines the attributes (like weights in Risk Assessor) and methods (like detect() or classify_objects()) that enable modular development. The connections indicate how data flows from detection through tracking to the final visualization output.

StartNode:

Represents the beginning of the activity workflow.Initiates the ROAD system processing sequence.

LoadVideo

Loads the input dashcam video file into the system.Prepares the video for frame-by-frame processing.

VideoLoaded?

Checks whether the video file is successfully loaded.If not loaded, the system stops or retries loading.

ReadNextFrame

Extracts the next frame from the video stream.Enables sequential frame-by-frame analysis.

FrameAvailable?:

Verifies if another frame exists in the video.If no frame is available, processing ends.

Detect Objects (YOLOv8Detector):

Identifies road objects such as vehicles and pedestrians.Generates bounding boxes, class labels, and confidence values.

TrackObjects (CentroidTracker):

Tracks detected objects across consecutive frames.Assigns unique IDs and determines motion patterns.

Analyze Lane Position (LaneDetector):

Divides frame into left, center, and right zones.Determines lane location of each tracked object.

ClassifyActions:

Determines object behavior based on motion and lane.Labels objects as approaching, departing, or stationary.

Save / Display Output:

Displays annotated frame to the user.Saves processed video with visual overlays.

Cleanup Resources:

Releases video capture and memory resources.Ensures proper termination of the system.

End Node:

Marks completion of activity flow.Stops system execution after processing finishes.

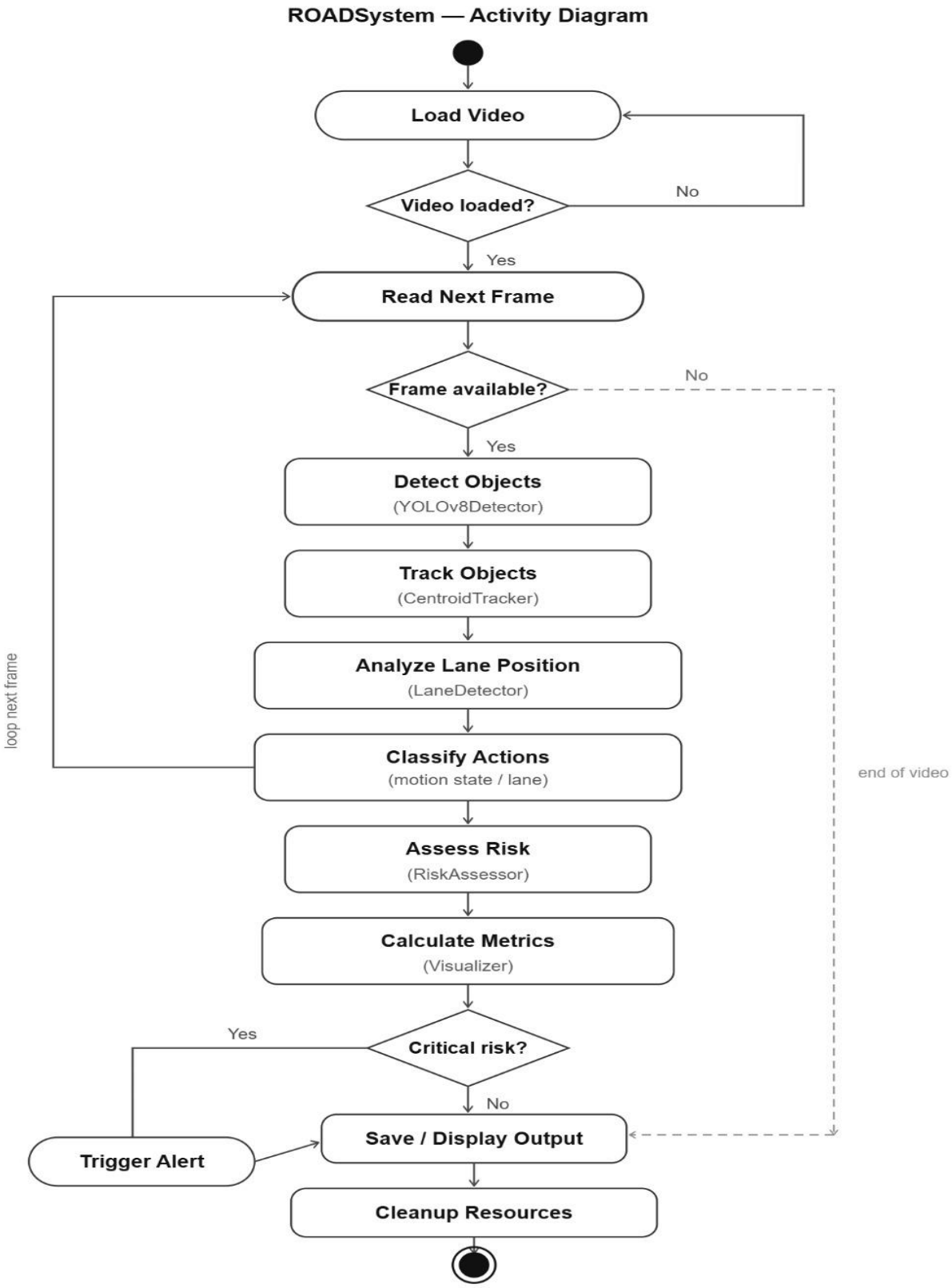


Fig 4.4 Activity Diagram

4.5 Sequence Diagram

To visualise the dynamic interaction and the chronological order of messages exchanged between objects during a single processing cycle. The diagram tracks a "Frame" as it moves from the VideoCapture source through the Detector for YOLO inference, then to the Tracker for ID assignment, and finally through the RiskEngine before being rendered.

It highlights the synchronous nature of the pipeline, ensuring that risk assessment only occurs after an object has been successfully detected, tracked, and placed within a specific lane. This ensures the integrity of the real-time safety alerts.

Actor (Driver/Admin):

Represents the external user interacting with the system.Initiates requests and receives processed results.

Lifeline:

Vertical dashed line representing the existence of an object over time.Shows interaction duration during sequence execution.

EventDetector (detector.py):

Detects objects from input frame using YOLO model.Returns list of detected bounding boxes and classes.

MotionTracker (tracker.py):

Tracks detected objects across frames using centroid logic.Updates object IDs and motion information.

EventAnalyzer (lane_detector):

Analyzes lane position of tracked objects.Assigns left, center, or right labels.

RiskAssessor (risk_assessor):

Calculates risk score for each detected object.Determines risk category based on weighted factors.

MetricsCalc (visualizer):

Generates an annotated frame with overlays.Calculates summary statistics and displays results.

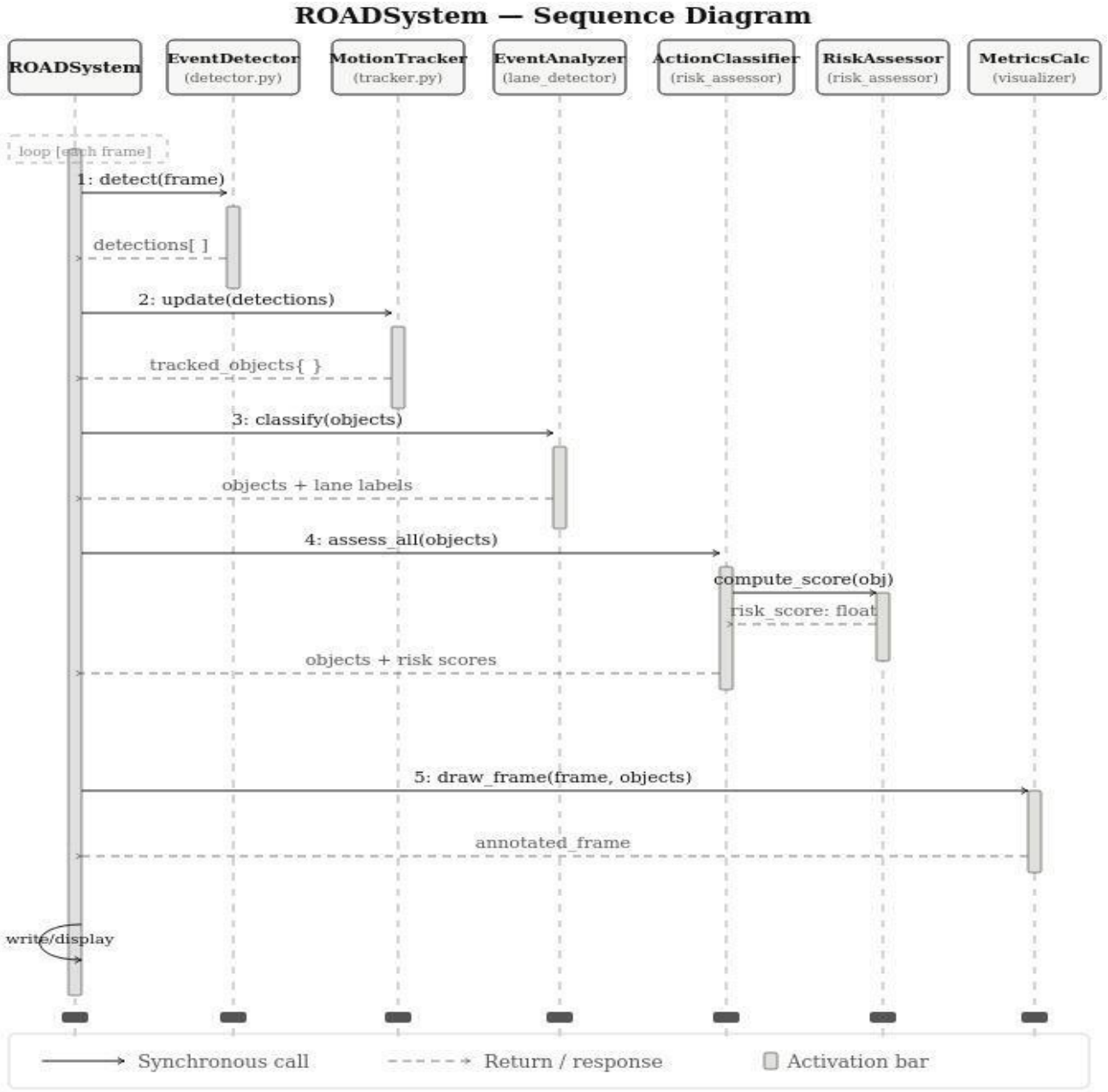


Fig 4.5 Sequence Diagram

4.6 System Architecture

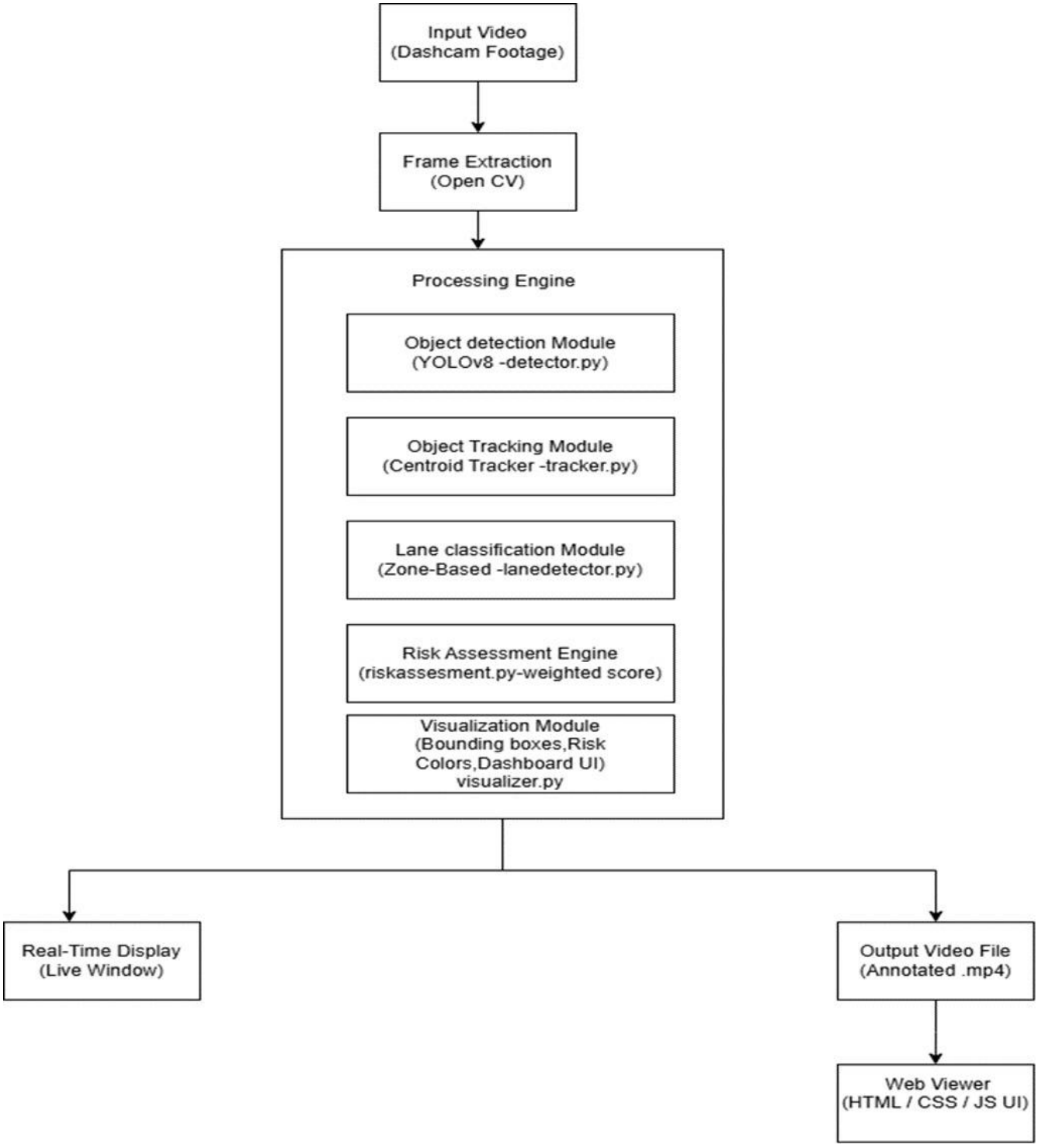


Fig 4.5 System Architecture Diagram

The ROAD system follows a sequential, modular pipeline designed for high-performance video processing. Each frame of the input video undergoes a multi-stage transformation to convert raw pixels into actionable safety intelligence.

FrameExtraction:

The process begins by reading the input video (typically in MP4 or AVI format) frame-by-frame using the OpenCV library. This stage acts as the data ingestion layer, ensuring that the video is decoded into a consistent BGR NumPy array format suitable for neural network inference. By managing the frame buffer and synchronization, this module ensures that the subsequent detection stages receive a steady stream of visual data at the correct aspect ratio and resolution.

Object Detector (YOLOv8):

Serving as the "eyes" of the system, this module utilizes the YOLOv8 Nano architecture, which is specifically optimized for high-speed real-time inference. To maintain high precision in a driving environment, the system filters the standard COCO dataset classes to focus exclusively on six road-relevant types: Cars, Trucks, Buses, Motorcycles, Bicycles, and Pedestrians. For every valid detection, the module extracts critical data including bounding box coordinates, class labels, and confidence scores, alongside spatial data such as the center point and the total pixel area of the detection.

ObjectTracker:

Rather than treating every frame as an isolated image, the Object Tracker links detections across time to provide temporal consistency. It assigns a unique persistent ID to each road user, allowing the system to track individual vehicles or pedestrians even if they are momentarily occluded. This module performs motion analysis by comparing the bounding box area across consecutive frames; an increasing area is classified as **APPROACHING**, a decreasing area as **DEPARTING**, and a steady area as **STATIONARY**, providing the system with a primitive but effective sense of depth and closing speed.

LaneDetector (Zone-based): .

This module provides the necessary spatial context relative to the vehicle's path by segmenting the frame into three vertical zones. The frame width is divided into a **Left Lane** (0% to 35%), a **Center Lane** (35% to 65%), and a **Right Lane** (65% to 100%). By mapping the center X-coordinate of every tracked object to these zones, the system identifies which hazards are directly in the driver's "Ego" path (the center lane). This classification is a high-priority signal that significantly influences the final risk calculations.

Risk Assessor (Weighted Scoring):

As the "brain" of the system, the Risk Assessor calculates a composite risk percentage using a sophisticated weighted formula. It balances four primary factors: **Distance (35%)** based on vertical positioning and box size, **Motion (30%)** which prioritizes approaching objects, **Lane Position (20%)** which flags objects in the center lane, and **Object Type (15%)** which accounts for the vulnerability of pedestrians and cyclists. The module then maps these scores to four distinct levels—Low, Medium, High, and Critical—preparing the data for real-time dashboard alerts and automated warning banners.

Output & Visualization:

The final module is responsible for rendering the processed intelligence back onto the visual stream. It applies color-coded bounding boxes and motion labels directly to the objects, draws vertical lane markers to visualize the safety zones, and generates a summary dashboard overlaying object counts and active critical warnings. The fully annotated stream is then exported as a new MP4 file and simultaneously displayed in a real-time window, providing the user with an intuitive and comprehensive safety interface.

4.7 Technology Description

The ROAD system utilizes a modern, high-performance technology stack tailored for real-time processing, modular scalability, and a seamless user experience. The integration of these core technologies ensures the system can handle the intensive computational demands of deep learning while remaining accessible and easy to maintain.

Python:

Python serves as the primary programming language for the entire system development. Its simple and readable syntax allows for a highly modular implementation, enabling the separate development and testing of the Detector, Tracker, and Risk Assessor modules. Beyond its ease of use, Python provides the critical glue for integrating industry-standard machine learning and computer vision libraries, ensuring that data flows efficiently from the raw video input to the final visual dashboard.

YOLOv8 (Ultralytics):

The system employs **YOLOv8 (You Only Look Once)**, a state-of-the-art real-time object detection model developed by Ultralytics. YOLOv8 is utilized to detect essential road entities, including cars, pedestrians, trucks, buses, and motorcycles. Unlike previous versions, YOLOv8 uses an **anchor-free detection** mechanism, which significantly improves accuracy and reduces the architectural complexity of the model. For every frame, it provides precise bounding box coordinates, class labels, and confidence scores, serving as the foundational "visual intelligence" of the project.

PyTorch:

PyTorch acts as the underlying deep learning framework and the primary backend for the YOLOv8 model. It provides the necessary tensor computation and neural network utilities required for high-speed inference. Most importantly, PyTorch supports **GPU acceleration (CUDA)**, which allows the system to offload heavy mathematical computations from the CPU to the Graphics Card. This hardware optimization enables the system to achieve the high frame rates (FPS) necessary for real-time driving safety applications.

OpenCV & Supporting Libraries:

In addition to the core AI stack, **OpenCV** is used for all image processing, video I/O, and real-time visualization tasks. **NumPy** is leveraged for high-speed numerical operations, particularly for calculating centroid distances and bounding box area changes. Together, these libraries facilitate the "Zone-based" lane detection and the weighted risk-assessment logic that defines the system's unique safety capabilities.

CHAPTER 5

IMPLEMENTATION & TESTING

5.1 Implementation

The implementation of the ROAD system follows a modular pipeline architecture. The description of each functional module and the dataset utilized are as follows:

Module Description:

Video Input & Frame Extraction: The system utilizes the **OpenCV** library to read video streams (standard .mp4 or .avi formats). It extracts frames sequentially at a rate defined by the source video or a central configuration.

Object Detection (YOLOv8): To identify and locate road entities in real-time.

Architecture: The system employs the **YOLOv8 nano** model, chosen for its balance between high inference speed and accuracy. It is pre- trained on the COCO dataset and filtered to focus on 6 target classes: *Person, Bicycle, Car, Motorcycle, Bus, and Truck*.
Output: Generates bounding box (bbox) coordinates, class labels, and confidence scores for every detected object.

Centroid Tracking: To maintain the identity of objects across multiple frames and analyze their movement.

Mechanism: Calculates the center point (centroid) of each bbox and matches it to existing IDs using Euclidean distance.

MotionAnalysis:

Tracks changes in the bounding box area over time. An increasing area indicates an APPROACHING object, while a decreasing area indicates a DEPARTING object.

LaneClassification:

Purpose: To provide spatial context for detected hazards.

Logic: Divides the frame width into three distinct zones based on percentage: **LEFT (0-35%)**, **CENTER (35-65%)**, and **RIGHT (65-100%)**. Objects are assigned a lane based on their horizontal centroid position.

Risk Assessment Engine: To calculate a quantitative risk score (0.0 to 1.0) for every tracked object.

Weighted Logic: Computes risk using five weighted factors: Proximity (Size), Motion State, Lane Position, Object Type, and Time-to-Collision (TTC).

Smoothing: Applies an **Exponential Moving Average (EMA)** to risk scores to prevent visual flickering caused by momentary detection gaps.

Visualization & UI: Rendering: Uses OpenCV to draw color-coded bounding boxes (Green, Yellow, Orange, Red) and a real-time "Safety Dashboard" overlay on the video.

Frontend Dashboard: A web interface built with **HTML/CSS/JS** that displays the project pipeline, technical stack, and a showcase of the analyzed safety footage.

Dataset Description: The model utilises weights derived from the **COCO (Common Objects in Context)** dataset, which is the industry standard for object detection.

1. **Scope:** Contains over 330,000 images with 80 object categories.
2. **Relevance:** The system specifically extracts road-relevant features to ensure high precision in identifying pedestrians and various vehicle types.
3. **Application:** By utilising a pre-trained backbone, the system benefits from deep feature hierarchies learned from millions of real-world instances.

5.2 Frontend Implementation

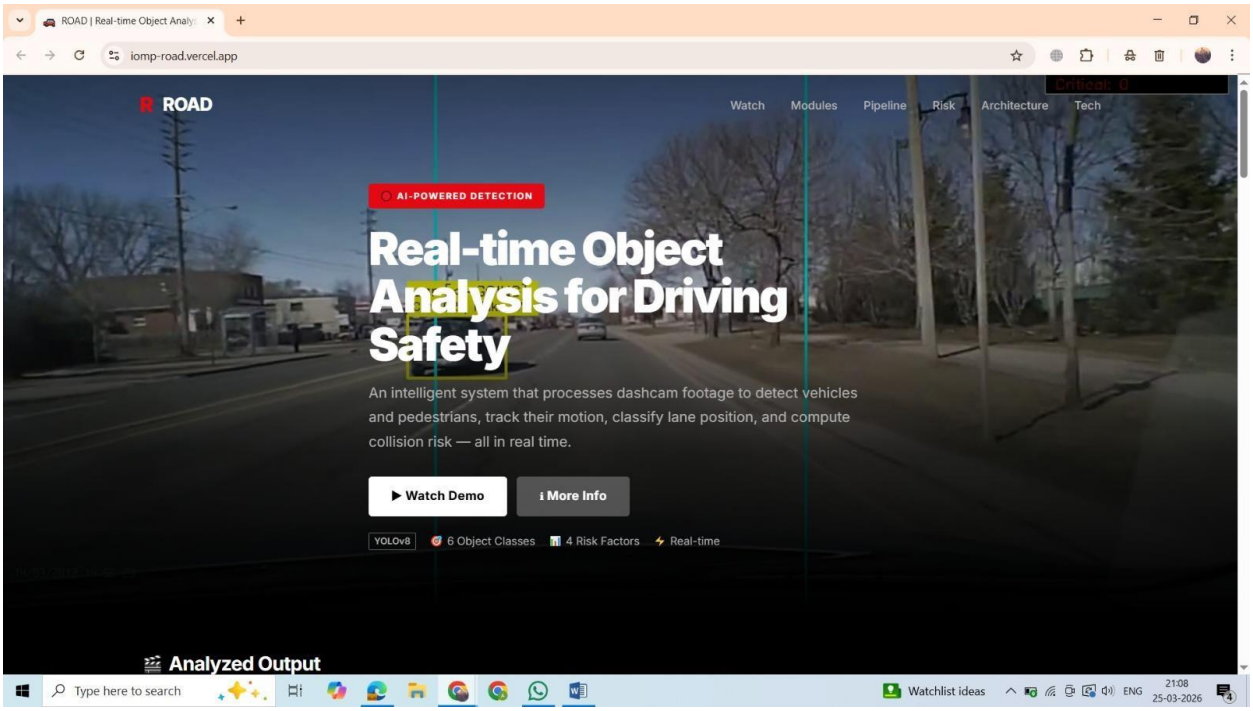


Fig 5.2.1 Home Page

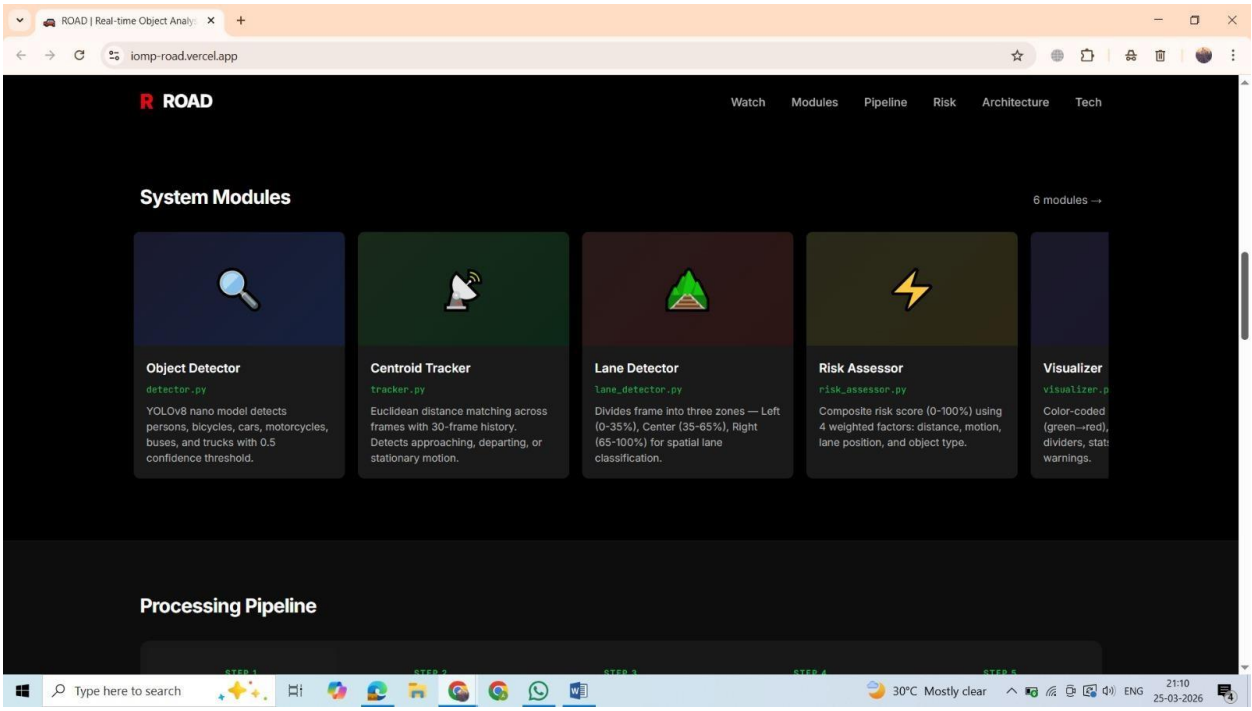


Fig 5.2.2 System Module

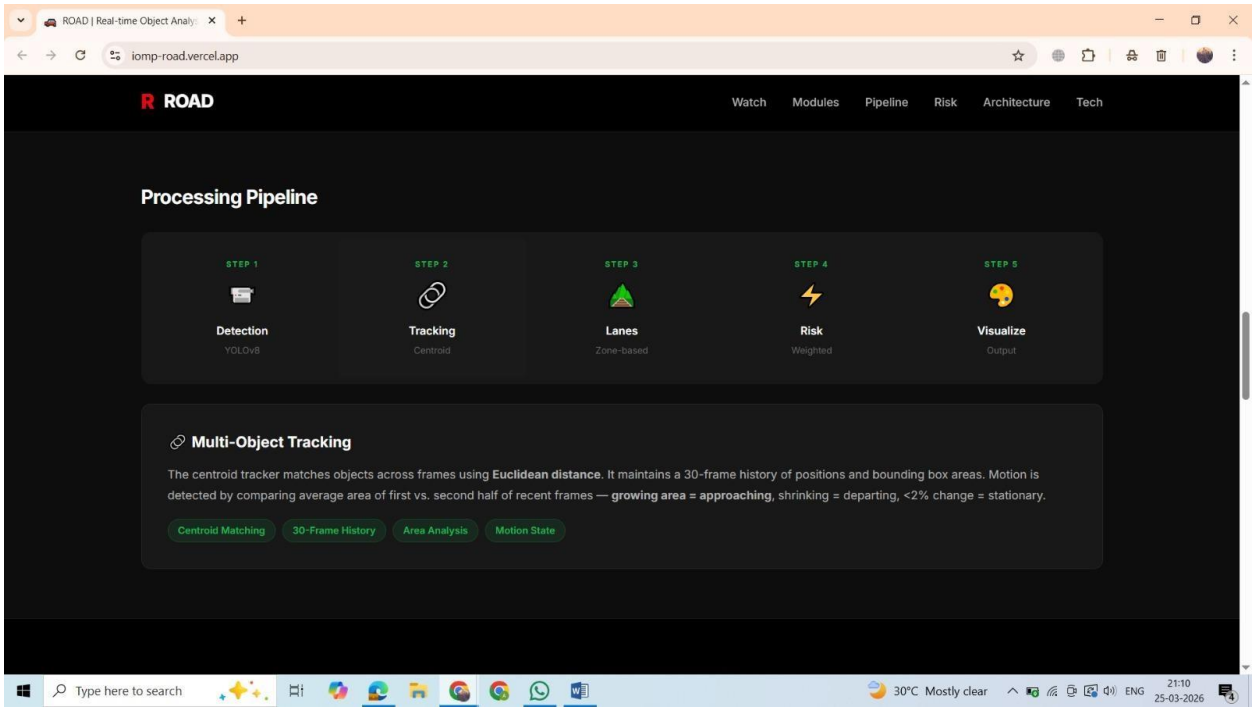


Fig 5.2.3 Pipe Line

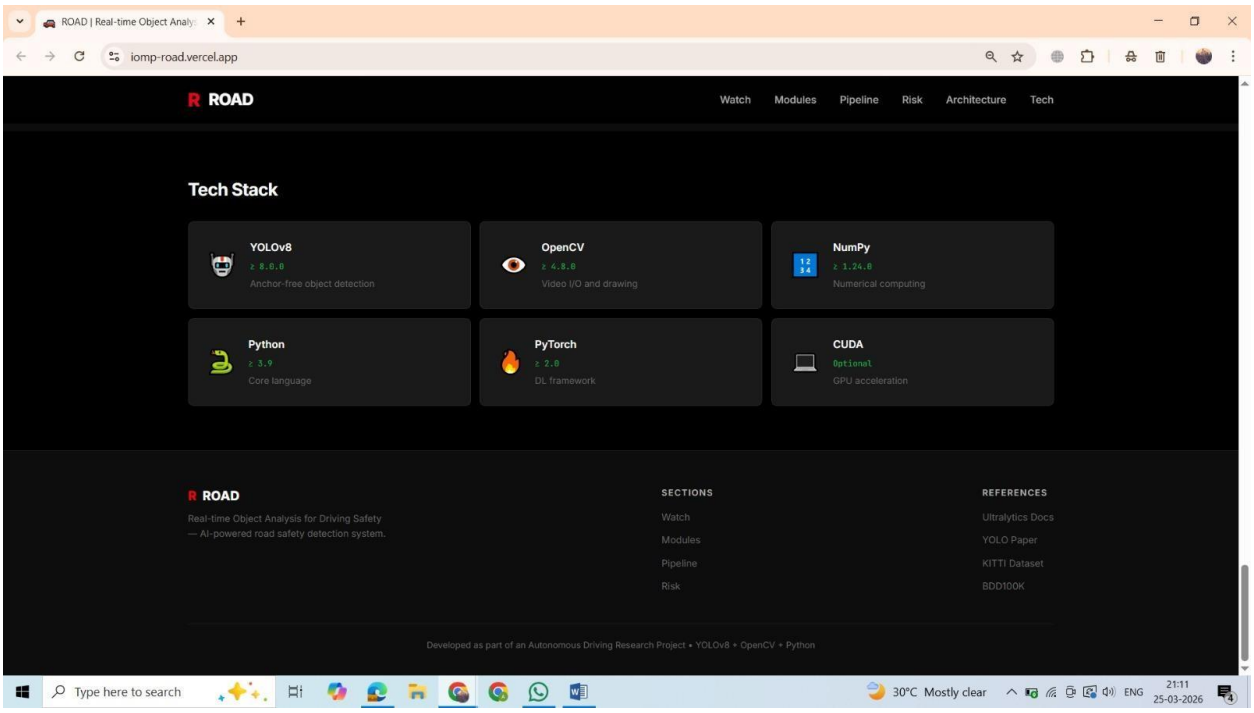


Fig 5.2.1 Tech Stack

5.3 Testing and Results

The system underwent rigorous functional testing using diverse dashcam video samples. The testing focused on the accuracy of object detection, the reliability of the centroid tracker, and the precision of the weighted risk-assessment engine.

Test Case 1: High-Speed Approaching Vehicle (Critical Risk)

Input Scenario: A vehicle in the CENTER lane rapidly increasing in size (bbox area).

System Logic: Lane Detector assigns CENTER lane.

Centroid Tracker detects positive area change → APPROACHING.

Risk Assessor calculates score > 0.75 due to Lane + Motion + Size weights.

Expected Result: Bounding box turns RED; Dashboard displays CRITICAL.

Actual Result: PASS. The system identified the hazard and maintained a stable alert using EMA smoothing.

Test Case 2: Pedestrian Proximity (High Risk)

Input Scenario: A pedestrian walking near the edge of the LEFT lane.

System Logic: Detector identifies class "person" (highest OBJECT_RISK multiplier: 1.0).

Risk Assessor applies the pedestrian weight.

Expected Result: Bounding box turns ORANGE; Dashboard displays HIGH.

Actual Result: PASS. Even at a distance, the higher weight assigned to vulnerable road users triggered an early warning.

Test Case 3: Stationary/Departing Traffic (Low Risk)

Input Scenario: A car in the RIGHT lane moving away from the camera.

System Logic: Centroid Tracker detects negative area change → DEPARTING.

Lane Detector assigns RIGHT lane (lower priority).

Expected Result: Bounding box remains GREEN; Dashboard displays LOW.

Actual Result: PASS. The system correctly filtered out non-threatening movement.

Test Case 4: Multi-Object Convergence (Multi-Boost)

Input Scenario: Two motorcycles approaching simultaneously in the CENTER lane.

System Logic: Risk Assessor triggers _apply_multi_object_boost.

Adds MULTI_OBJECT_BOOST (+0.10) to both individual scores.

Expected Result: Risk level escalates from Medium to HIGH/CRITICAL due to restricted escape routes.

Actual Result: PASS. The additive risk logic accurately reflected the increased danger of a crowded path.

5.4 Summary Table of Test Execution

Test Case	Scenario	Key System Logic	Expected Result	Actual Result	Status
Test Case 1: High-Speed Approaching Vehicle (Critical Risk)	Vehicle in CENTER lane rapidly approaching	Lane: CENTER; Motion: APPROACHING; Risk > 0.75	RED box; CRITICAL alert	Hazard detected; stable alert (EMA smoothing)	PASS
Test Case 2: Pedestrian Proximity (High Risk)	Pedestrian near LEFT lane edge	Detected: person; Highest risk weight applied	ORANGE box; HIGH alert	Early warning triggered	PASS
Test Case 3: Stationary/Departing Traffic (Low Risk)	Car in RIGHT lane moving away	Motion: DEPARTING; Lane: RIGHT (low priority)	GREEN box; LOW alert	Non-threatening movement ignored	PASS
Test Case 4: Multi-Object Convergence (Multi-Boost)	Two motorcycles approaching in CENTER lane	Multi-object boost applied (+0.10)	HIGH/CRITICAL risk	Risk increased due to crowded path	PASS

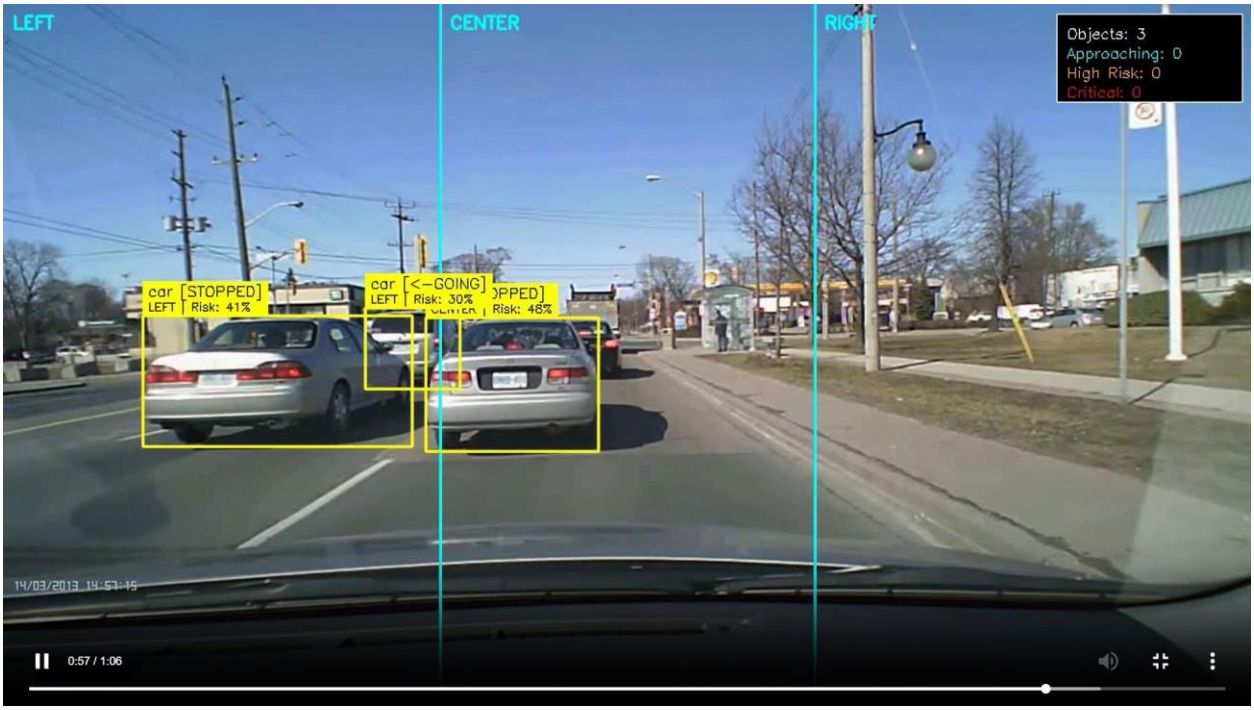


Fig 5.4.2 Output(The image shows it has detected the vechiles with are moving on its left and center lanes)

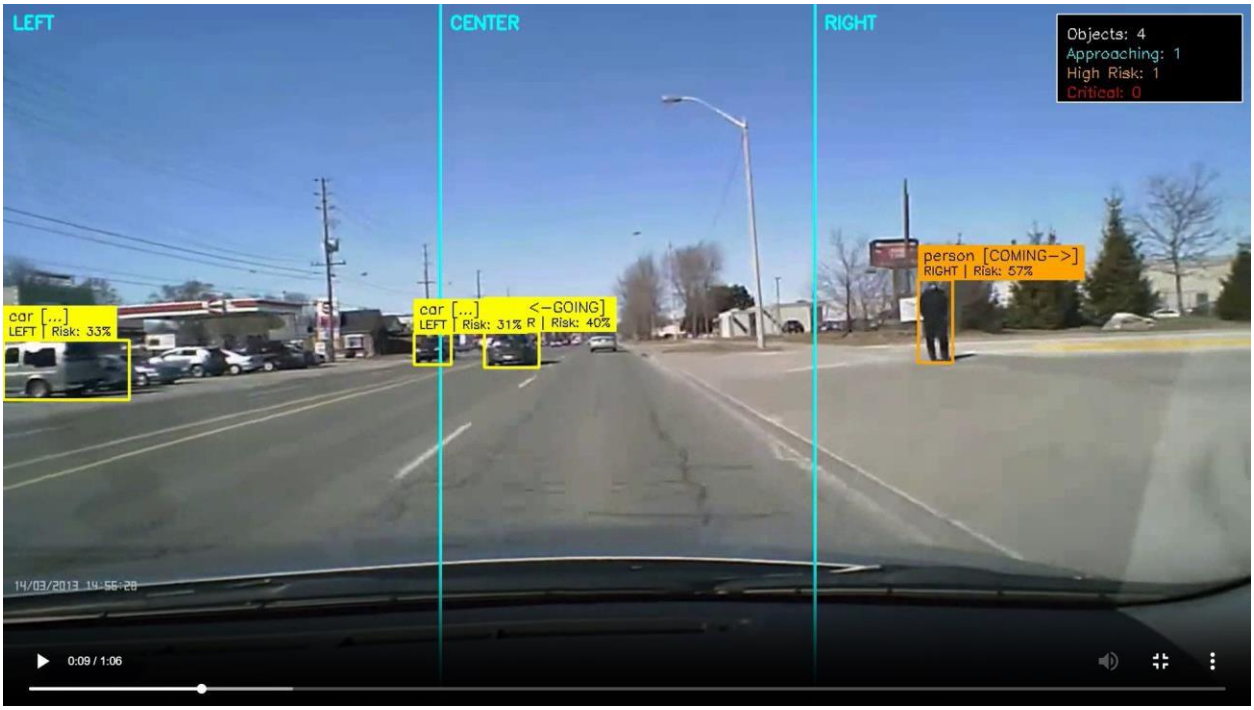


Fig 5.4.2 Output(The image shows it has detected the human on the right lane and gives him a high risk, while the other moving vehicles have a lower Risk)

CHAPTER 6

CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The **ROAD (Real-time Object Analysis for Driving Safety)** project successfully demonstrates the integration of deep learning and computer vision to enhance vehicular safety. By leveraging the **YOLOv8** architecture, the system achieves high-speed object detection capable of identifying critical road entities such as pedestrians, vehicles, and cyclists from standard dashcam footage.

The core innovation of this project lies in its **Multi-Factor Risk Assessment Engine**, which moves beyond simple detection by incorporating spatial awareness (Lane Classification) and temporal dynamics (Motion Tracking).

The implementation of **Exponential Moving Average (EMA)** for score smoothing and **Multi-Object Boost** logic ensures that the system provides reliable, non- flickering alerts even in complex traffic scenarios. The final output—a color-coded visual interface and a web-based dashboard—proves that sophisticated AI safety logic can be translated into intuitive, actionable feedback for a driver. Ultimately, this project serves as a robust, vision-only foundation for Advanced Driver- Assistance Systems (ADAS) that do not rely on expensive LiDAR or Radar hardware.

6.2 Future Scope

While the current system provides a functional safety framework, the following enhancements are proposed to move the project toward a production-ready autonomous environment:

Deep-Learning Lane Detection: Replace the current static zone-based lane division with a dynamic segmentation model (e.g., **SCNN** or **LaneNet**) to handle curved roads, lane changes, and varying camera angles.

Time-to-Collision (TTC) Precision: Integrate camera calibration and "depth- from-mono" estimation to calculate exact physical distances (in meters) and precise TTC in seconds, rather than percentage-based risk.

Audio-Haptic Alerts: Implement a localized audio warning system that uses spatial audio to alert the driver to the specific direction of an approaching hazard.

Edge Deployment: Optimize the model using **TensorRT** or **OpenVINO** for deployment on edge computing devices like NVIDIA Jetson or Raspberry Pi for real-time in-vehicle use.

Night Vision & Weather Robustness: Fine-tune the detection model on datasets specifically featuring low-light, rainy, or foggy conditions to ensure reliability across all driving environments.

V2X Integration: Explore Vehicle-to-Everything (V2X) communication, allowing the system to receive data from smart traffic lights and other connected vehicles to predict hazards beyond the camera’s line of sight.

References

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012. [Online]. Available: http://www.cvlibs.net/datasets/kitti/

[2] T. Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014. [Online]. Available: https://cocodataset.org/

[3] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking (SORT)," in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2016. [Online]. Available: https://arxiv.org/abs/1602.00763

[4] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2017. [Online]. Available: https://arxiv.org/abs/1703.07402

[5] C. R. Harris *et al.*, "Array Programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020. [Online]. Available: https://numpy.org/doc/

[6] F. Yu *et al.*, "BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. [Online]. Available: https://bdd-data.berkeley.edu/

[7] Ultralytics, "YOLOv8: State-of-the-Art Object Detection and Predictive Model," 2023. [Online]. Available: https://docs.ultralytics.com

[8] PyTorch Developers, "PyTorch Documentation," 2023. [Online]. Available: https://pytorch.org/docs

[9] OpenCV Team, "Open Source Computer Vision Library (OpenCV) Documentation," 2023. [Online]. Available: https://docs.opencv.org/

[10] World Health Organization, "Global Status Report on Road Safety 2023," 2023. [Online]. Available: https://www.who.int/publications/i/item/global-status-report-on-road-safety

[11]. ROAD THE ROAD EVENT AWARENESS DATASET FOR AUTONOMOUS DRIVING Abstract: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9712346

Appendix: (Sample source code)

```
import cv2
import os
import sys
import subprocess
import webbrowser

def get_video_path():
    """Get video path from command line or use default."""
    for i, arg in enumerate(sys.argv):
        if arg in ('--video', '-v') and i + 1 < len(sys.argv):
            return sys.argv[i + 1]
    return 'input.mp4'

def run_analysis(video_path):
    """Step 1: Run main.py to analyze the video."""
    print("=" * 60)
    print("STEP 1: Running road safety analysis...")
    print("=" * 60)
    result = subprocess.run(
        [sys.executable, 'main.py', '--video', video_path, '--no-display'],
        cwd=os.path.dirname(os.path.abspath(__file__))
    )
    if result.returncode != 0:
```

```
print("ERROR: Analysis failed!")
ys.exit(1)

def reencode_for_website(video_path):
    """Step 2: Re-encode analyzed video to H.264 for browser playback."""
    print("\n" + "=" * 60)
    print("STEP 2: Re-encoding video for website...")
    print("=" * 60)

    base = os.path.splitext(video_path)[0]
    analyzed_path = f"{base}_analyzed.mp4"
    output_path = "input_analyzed_h264.mp4"

    if not os.path.exists(analyzed_path):
        print(f"ERROR: Analyzed video not found: {analyzed_path}")
        sys.exit(1)

    # Get properties from analyzed video
    cap_analyzed = cv2.VideoCapture(analyzed_path)
    analyzed_frames = int(cap_analyzed.get(cv2.CAP_PROP_FRAME_COUNT))
    # Downscale by 50% to ensure file size stays under GitHub's 100MB limit
    orig_w = cap_analyzed.get(cv2.CAP_PROP_FRAME_WIDTH)
    orig_h = cap_analyzed.get(cv2.CAP_PROP_FRAME_HEIGHT)
    w = int(orig_w * 0.5)
    h = int(orig_h * 0.5)
    cap_analyzed.release()
```

```
# Get original video duration to calculate correct FPS
cap_input = cv2.VideoCapture(video_path)
input_frames = int(cap_input.get(cv2.CAP_PROP_FRAME_COUNT))
input_fps = cap_input.get(cv2.CAP_PROP_FPS)
input_duration = input_frames / input_fps if input_fps > 0 else 60
cap_input.release()

# Calculate correct FPS so duration matches original
correct_fps = analyzed_frames / input_duration if input_duration > 0 else 16
correct_fps = max(1, min(correct_fps, 60))  # Clamp to reasonable range

print(f"  Analyzed video: {analyzed_frames} frames, {w}x{h}")
print(f"  Original duration: {input_duration:.1f}s")
print(f"  Using FPS: {correct_fps:.1f}")

# Re-encode to H.264
cap = cv2.VideoCapture(analyzed_path)
fourcc = cv2.VideoWriter_fourcc(*'avc1')
writer = cv2.VideoWriter(output_path, fourcc, correct_fps, (w, h))

if not writer.isOpened():
    # Fallback codec
    fourcc = cv2.VideoWriter_fourcc(*'XVID')
    output_path = "input_analyzed_h264.avi"
```

```
        writer = cv2.VideoWriter(output_path, fourcc, correct_fps, (w, h))

    count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame_resized = cv2.resize(frame, (w, h))
        writer.write(frame_resized)
        count += 1
        if count % 200 == 0:
            print(f"  {count}/{analyzed_frames} frames...")

    cap.release()
    writer.release()

    duration = count / correct_fps
    size_mb = os.path.getsize(output_path) / (1024 * 1024)
    print(f"  Done! {count} frames → {duration:.1f}s ({size_mb:.1f} MB)")
    print(f"  Saved to: {output_path}")

def open_website():
    """Step 3: Open the website in default browser."""
    print("\n" + "=" * 60)
    print("STEP 3: Opening website...")
```

```
    print("=" * 60)
    html_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"index.html")
    webbrowser.open(f"file:///{html_path}")
    print(f"  Opened: {html_path}")


if __name__ == "__main__":
    video_path = get_video_path()
    print(f"\n🚗 ROAD - Website Updater")
    print(f"    Input video: {video_path}\n")

    run_analysis(video_path)
    reencode_for_website(video_path)
    open_website()
```

PLAGIARISM REPORT

Chapter No.	Chapter Name	Number of Words	Plagiarism
1	Abstract	156	9%
2	Introduction	745	7%
3	Literature Survey	322	12%
4	Software & Hardware specifications	612	7%
5	Proposed System Design	1185	9%
6	Implementation & Testing	860	0%
7	Conclusion and Future scope	420	6%
	Total	4282	